BrailleLens

Visione Artificiale

Salvatore Bennici

A.A. 2025-2026

Contents

1 Introduzione **3**

2 Stato Dell'Arte **5**
2.1 Dall'Image Processing al Deep Learning 5
2.2 Approcci basati su Object Detection 5
2.3 Approcci basati su Segmentazione Semantica 6
2.4 Robustezza all'Illuminazione 6
2.5 Architetture Leggere per Dispositivi Mobili 6

3 Approccio Sviluppato **8**
3.1 Il dataset 8
3.1.1 I limiti del dataset 9
3.1.2 Il dataset completo 9
3.2 L'approccio iniziale 11
3.2.1 Il Detector 13
3.2.2 Il classificatore 15
3.3 Il modello finale 19

4 Valutazione Sperimentale **20**
4.1 Valutazione del detector 21
4.2 Valutazione del classificatore 23
4.3 La valutazione complessiva 25
4.3.1 Benchmarking di Riferimento (Reference SOTA) 26

5 Conclusioni **28**

1 Introduzione

Il problema affrontato nel progetto è quello del riconoscimento di testo Braille o OBR (Optical Braille Recognition). L'idea alla base è quella di creare uno strumento in grado di rilevare e riconoscere del testo Braille. In particolare, si pone il proposito di utilizzare un'architettura leggera, adatta all'utilizzo su dispositivi mobili, e conseguentemente abbastanza performante da ottenere risultati accurati anche partendo da immagini "in the wild", quindi scattate in condizioni non ottimali come possono essere quelle ottenute da uno smartphone.

Il Braille è un metodo di scrittura tattile, concepito per essere letto con le dita e non con gli occhi. Quindi non è un segno cromatico su una superficie, come può esserlo un tratto di penna o del testo stampato, ma una serie di piccoli rigonfiamenti puntiformi. Ogni carattere è formato da uno a sei "punti" disposti rigidamente su un'astratta griglia di dimensioni 3×2. Questa struttura fisica comporta sfide significative, ma offre anche un forte vincolo geometrico che aiuta a definire la struttura spaziale del problema. Le principali sfide affrontate includono:

- **Basso contrasto locale:** I rilievi Braille generano gradienti di intensità molto deboli rispetto al background, rendendo complessa l'estrazione delle feature discriminanti in presenza di rumore visivo o tessiture superficiali irregolari.

- **Elevata densità spaziale:** Le dimensioni ridotte e la prossimità dei punti richiedono modelli con un'alta risoluzione spaziale; piccoli errori di localizzazione possono portare a una classificazione errata dell'intera cella.

- **Dipendenza dalla sorgente luminosa:** La visibilità dei punti è legata quasi esclusivamente alle ombre portate. Variazioni nell'angolo di incidenza della luce alterano drasticamente la morfologia percepita, rappresentando una sfida per l'invarianza del modello.

- **Distorsioni geometriche e prospettiche:** Sebbene la struttura del carattere sia rigidamente definita, fattori come la curvatura del supporto, le rotazioni dell'immagine o distorsioni prospettiche possono compromettere la geometria del pattern.

- **Rumore strutturato (Interferenza recto-verso):** Un testo Braille su carta è solitamente scritto su pagine fronte-retro, quindi saranno visibili sia le convessità dei punti da identificare, che le concavità di quelli che compongono il testo della pagina retrostante. Questo genera uno sfondo rumoroso con caratteristiche morfologiche estremamente simili ai target da rilevare (le convessità).

- **Eterogeneità dei contesti:** Al di fuori del supporto cartaceo, i caratteri Braille compaiono spesso su superfici non convenzionali (metalli, plastiche, tastiere), richiedendo capacità di generalizzazione elevate per operare su sfondi complessi.

2 Stato Dell'Arte

La letteratura scientifica riguardante l'Optical Braille Recognition (OBR) ha subito una profonda evoluzione nell'ultimo decennio, spostandosi da tecniche di elaborazione delle immagini tradizionali verso approcci basati sul Deep Learning. Questa transizione è stata guidata dalla necessità di superare le limitazioni dei metodi geometrici classici, che risultavano inefficaci in condizioni non controllate, come evidenziato in recenti survey [3].

2.1 Dall'Image Processing al Deep Learning

I primi sistemi OBR si basavano pesantemente sulla regolarità della griglia Braille, utilizzando tecniche come adaptive thresholding, il rilevamento dei contorni (Canny Edge Detection) e filtri di Gabor per identificare i punti. Tuttavia, come notato da Mahale et al. [3], questi approcci soffrono significativamente in presenza di rotazioni, deformazioni della carta o rumore di fondo (es. macchie o texture). L'avvento delle Convolutional Neural Networks (CNN) ha permesso di apprendere feature più robuste, gestendo meglio la variabilità intrinseca delle immagini acquisite da fotocamera. A tal proposito, uno dei lavori più iconici è quello proposto da Ovodov et al.[4].

2.2 Approcci basati su Object Detection

Una delle correnti di ricerca più promettenti applica algortimi di Object Detection generici al riconoscimento del Braille. Dove si pone come obiettivo quello di localizzare le singole celle Braille (a volte anche porzioni di testo) all'interno dell'immagine e classificarle. Studi recenti hanno dimostrato l'efficacia della famiglia di modelli YOLO (You Only Look Once) per questo scopo.

Valencio e Rachmawanto [7] hanno proposto un sistema a due stadi: il primo utilizza *YOLOv8* per localizzare le regioni di interesse contenenti caratteri Braille, mentre il secondo stadio impiega una rete *MobileNetV2* per la classificazione dei caratteri ritagliati. Questo approccio disaccoppiato permette di ottimizzare indipendentemente la localizzazione e il riconoscimento, ottenendo elevate prestazioni anche su dispositivi con risorse limitate.

Parallelamente, Sun et al. [6] hanno presentato un miglioramento mirato su *YOLOv11* per la rilevazione in "scene naturali". Il loro lavoro affronta

specificamente il problema dei piccoli oggetti (i punti Braille) su sfondi complessi. Introducendo moduli di attenzione specifici come ULSAM (Ultra-Lightweight Subspace Attention Module) e usando loss function specifiche (SDIoU), sono riusciti a migliorare drasticamente la distinzione tra punti Braille e rumore di fondo (come texture di tessuti o riflessi metallici), un problema critico per le applicazioni reali.

2.3 Approcci basati su Segmentazione Semantica

Un approccio alternativo alla rilevazione degli oggetti è la segmentazione semantica, che tenta di classificare ogni pixel dell'immagine. Li et al. [2] hanno introdotto *BraU-Net*, un'architettura basata su U-Net. Invece di imporre griglie rigide, BraU-Net apprende la struttura globale della pagina e segmenta i caratteri Braille a livello di pixel. Questo metodo si è rivelato particolarmente efficace nel gestire le deformazioni geometriche del foglio (curvature). Inoltre, l'uso di task ausiliari per l'estrazione del foreground aiuta a ridurre il rumore proveniente dai punti presenti sulla pagina retrostante (problema del double-sided Braille).

2.4 Robustezza all'Illuminazione

Come anticipato nell'introduzione, la dipendenza dalle ombre è uno dei talloni d'Achille dell'OBR. Yamashita et al. [8] hanno affrontato esplicitamente questo problema proponendo modelli indipendenti dalla direzione della luce. Confrontando architetture come RetinaNet, YOLOX e BraU-Net su un dataset appositamente creato con illuminazione variabile a 360 gradi, hanno dimostrato che, con un adeguato addestramento e data augmentation, le reti neurali moderne possono imparare a riconoscere i pattern di punti indipendentemente da come le ombre vengono proiettate, superando i vincoli delle vecchie tecniche che richiedevano luce radente specifica.

2.5 Architetture Leggere per Dispositivi Mobili

Dato che uno degli obiettivi che ci si pone in questo progetto è l'esecuzione su dispositivi mobili, è fondamentale analizzare lo stato dell'arte riguardante le reti "lightweight". Oltre al già citato utilizzo di MobileNetV2 da parte di Valencio [7], dalla ricerca della lettaratura effettuata non sono emerse particolari aprrocci per questo contesto. Seppur per pressupposti diversi vi sono

alcuni esperimenti interessanti come quello di Sayeedi et al. [5] dove hanno proposto *BrailleNet*, una CNN personalizzata e leggera progettata per essere eseguita su hardware edge come il Raspberry Pi. Il loro sistema, integrato in un prototipo di guanto assistivo ("BrailleSense"), dimostra che è possibile ottenere un'accuratezza elevata (oltre il 97%) con modelli computazionalmente economici, aprendo la strada ad applicazioni OBR portatili e in tempo reale che non dipendono da server remoti.

In sintesi, lo stato dell'arte attuale si sta muovendo decisamente verso modelli *single-stage* veloci (come le varianti avanzate di YOLO) o architetture di segmentazione robuste, con una crescente attenzione all'efficienza computazionale (MobileNet, reti custom) per abilitare l'assistenza in tempo reale agli utenti ipovedenti.

3 Approccio Sviluppato

Il progetto verte sull'implementazione di un modello per il rilevamento e la classificazione di caratteri Braille che funzioni in ambienti diversi e su dispositivi mobili, anche poco prestanti, e che quindi risulti efficace ma computazionalmente leggero. Per allenare tale modello è stato utilizzato un dataset composito personalizzato formato da tre dataset noti in letteratura, di cui due raffiguranti immagini su carta e un terzo con immagini "in the wild".

Le sezioni che seguono presentano il dataset utilizzato, le reti neurali adottate e una panoramica sul modello completo.

3.1 Il dataset

Il primo dataset ad essere preso in considerazione è stato AngelinaDataset [4]. Esso contiene una raccolta di circa 300 fotografie di pagine Braille scattate tramite diverse fotocamere e smartphone in ambiente tendenzialmente controllato, con illuminazione proveniente dalla parte superiore o superiore sinistra. Le pagine raffigurate provengono per la maggior parte da libri di testo aperti, e come conseguenza presentano naturali curvature che vanno da nulle o lievi, a moderate.

In particolare il dataset contiene:

- 212 pagine fronte-retro provenienti da libri in Braille;
- 22 lavori scritti a mano da studenti;
- 44 fotografie non in Braille di documenti vari reperiti su Internet utilizzate come fonte ausiliaria di contro esempi;
- 50 pagine in Braille provenienti da altri libri e manoscritti (usati come test set esterno nel lavoro di Ovodov [4]).

Le labels originali constano di 4 parametri per la definizione della bounding box di ogni carattere Braille e di un'etichetta, che ne rappresenta la classe, che assume valori in base a quali dei 6 punti sono presenti, per un totale di 64 combinazioni possibili. Tuttavia, è importante precisare che alcune di queste classi non compaiono mai all'interno del dataset, e che la distribuzione di quelle presenti è fortemente sbilanciata, con migliaia di istanze per alcune, appena qualche unità per altre, e valori intermedi per la

maggior parte. Sebbene questa caratteristica sia perfettamente normale e tipica di un qualsiasi linguaggio poiché certi caratteri risultano naturalmente più utilizzati di altri, pone un problema in ambito di riconoscimento basato sul deep learning, in quanto i caratteri assenti o scarsamente rappresentati non potranno essere efficacemente appresi dai modelli.

La suddivisione in train e validation set è operata sul dataset direttamente dagli autori originali, è randomizzata e cerca di mantenere la proporzione fra le classi; reputando tale metodo appropriato ai fini del progetto si è scelto di mantenere inalterata la divisione.

3.1.1 I limiti del dataset

Come accennato, le fotografie nel dataset Angelina sono state scattate in ambiente tendenzialmente controllato, e provengono, per la maggior parte, da libri di testo. Questo comporta una critica monodimensionalità del dataset, ovvero le immagini sono molto simili tra loro, per struttura e contenuto: l'inquadratura ritrae sempre una pagina, lo sfondo è cartaceo, l'illuminazione proveniente dalla medesima angolazione.

Questo può portare un modello di riconoscimento a confondersi nel momento in cui gli vengono proposte immagini dissimili a quelle su cui è stato allenato.

3.1.2 Il dataset completo

Nel tentativo di ovviare a questa problematica è stato aggiunto un secondo dataset, più variegato e con varie rappresentazioni "in the wild". Il dataset Natural Scene è stato agglomerato al dataset Angelina, con lo scopo di generarne uno più ampio, più vario, e meno prono a problemi di overfitting da parte dei modelli che lo utilizzano per l'allenamento.

Anche in questo caso la divisione del dataset in train, validation e test è stata operata direttamente dagli autori mantenendo una buona proporzione tra le classi.

Il difetto principale di questo dataset è la dimensione estremamente ridotta, specialmente se rapportato al primo. Per ovviare al problema si è fatto ricorso all'oversampling: il train set di Natural Scene è stato clonato cinque volte, raggiungendo così circa il 20% dei caratteri presenti nel dataset completo. Tali valori sono stati scelti per evitare che le immagini venissero

interpretate come rumore dal modello, ma evitando di perdere generalità a causa di eventuale memorizzazione dovuta a un'eccessiva replicazione.

A questi è stato successivamente aggiunto il dataset DSBI (usato per confronto anche in [4]), molto usato in letteratura e strutturalmente simile al dataset Angelina: comprende fotografie scattate in ambiente controllato ritraenti Braille su carta, con illuminazione prevalentemente dall'alto, inoltre soffre dei medesimi problemi e in maniera più incisiva.

Anche in questo caso la divisione del dataset è operata dagli stessi autori: il test set comprende molte immagini provenienti da libri non presenti nel train set e risulta, nel complesso, molto più grande di quest'ultimo.

La scelta di agglomerarlo ai primi due è di natura pratica, poiché si è osservato che il modello riesce a generalizzare meglio quando allenato su tutti e tre.

Il dataset risultante è stato convertito in struttura standard YOLO, formato richiesto dal detector.

Le tavole che seguono mostrano una panoramica della conformazione del dataset completo.

Ulteriori analisi esplorative sul dataset sono disponibili nel Jupyter Notebook `01_EDA.ipynb`. Inoltre, è stato predisposto uno script di navigazione e visualizzazione situato in `src/data/explore_dataset.py`, il quale sfrutta le funzionalità del framework FiftyOne.

	Images				**Characters**			
Dataset	**Train**	**Val**	**Test**	**Total**	**Train**	**Val**	**Test**	**Total**
Angelina	225	46	50	321	55.470	13.528	13.013	82.011
NaturalScene	740	185	232	1.157	3.321	840	979	5.140
DSBI	42	10	176	228	15.226	4.809	71.220	91.255
Merged	3.771	236	370	4.377	78.862	16.734	49.636	145.232

Table 1: Composizione del dataset

Figure 1: Distribuzione delle classi nel set di training.

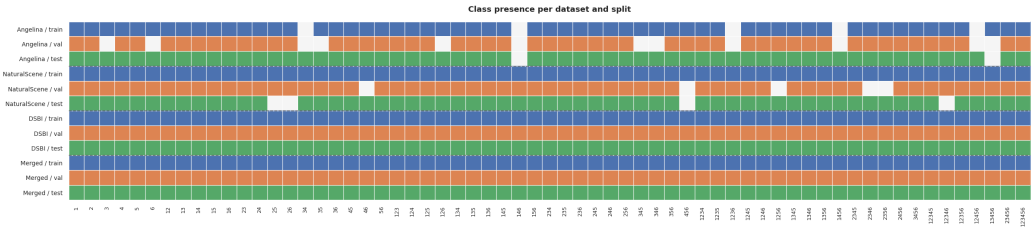

Figure 2: Presenza delle classi nei vari dataset

3.2 L'approccio iniziale

Prendendo come esempio il lavoro di Ovodov [4], si è inizialmente pensato di riadattare tale approccio alle esigenze del progetto.

Nell'articolo viene utilizzato un modello pre-addestrato RetinaNet end-to-end che mira a riconoscere ogni carattere come appartenente a una delle 64 classi possibili. Questo modello, seppur dimostra ottimi risultati, risulta pesante e con un costo computazionale elevato che mal si adegua a un utilizzo su dispositivi mobili.

Dovendo virare su un approccio più leggero che mantenesse buoni livelli di performance la scelta è ricaduta su un'architettura della famiglia YOLO, ampiamente considerata in letteratura come un framework leggero e affidabile nel campo dell'Optical Braille Recognition (OBR). Reti come Faster R-CNN,

seppur paragonabili a livello di accuratezza, risultano infatti inadatte per applicazioni mobili e real-time a causa dell'elevata latenza computazionale. Nello specifico, si è optato per l'adozione di YOLO26n (Ultralytics), una rete in versione *nano* da circa 2 milioni di parametri, dimensionata in modo ottimale per le stringenti esigenze del progetto.

Questa versione introduce novità determinanti per la massimizzazione delle performance su *micro-target*. In particolare, l'implementazione del meccanismo di assegnazione delle etichette STAL (*Small-Target-Aware Label Assignment*), coadiuvato dalla funzione di costo *ProgLoss* (*Progressive Loss Balancing*), impedisce che i gradienti derivanti dai target più grandi o dal background dominino l'addestramento. Tale approccio garantisce che le *feature* minuscole e a basso contrasto, tipiche dei singoli punti Braille, ricevano una supervisione calibrata. A livello di ottimizzazione, la stabilità del training è garantita dall'impiego dell'algoritmo *MuSGD*, un ottimizzatore ibrido nativo di YOLO26 che spesso di rivela adeguato quando si opera su dataset caratterizzati da pattern così densi, piccoli e ripetitivi.

Inoltre, per ottimizzare l'efficienza rispetto alle generazioni precedenti, YOLO26 adotta un design *End-to-End NMS-Free*. Addestrando il modello a generare predizioni univoche, viene eliminato il costoso step di post-elaborazione noto come *Non-Maximum Suppression* (NMS). Questa innovazione, unita alla rimozione del modulo DFL (*Distribution Focal Loss*), semplifica notevolmente il grafo computazionale, garantendo un'inferenza su CPU fino al 43% più rapida rispetto alla famiglia YOLO11 e agevolando il *deployment* su dispositivi edge e smartphone.

Inizialmente, si è esplorata la possibilità di impiegare YOLO26n in una configurazione *single-stage*, delegando al medesimo modello sia la localizzazione spaziale che la classificazione dei caratteri Braille. Tuttavia, i risultati empirici hanno evidenziato i limiti di questo approccio su un task così fortemente sbilanciato sui dettagli minuti. Di conseguenza, si è scelto di riformulare il problema disaccoppiando le due operazioni attraverso un'architettura modulare basata su una *two-stage pipeline*, in modo simile come fatto da Valenctio et al. [7].

Nel primo stadio, YOLO26n opera esclusivamente come *detector*, concentrando la propria capacità rappresentativa sull'individuazione delle singole celle Braille. Nel secondo stadio, le regioni così isolate vengono processate da un classificatore dedicato, implementato tramite una CNN custom. Questa separazione architetturale permette alla CNN di specializzarsi unicamente nell'apprendimento delle feature topologiche intra-cella, svincolandola dalle

complessità legate al rumore di fondo e garantendo una classificazione finale più robusta e precisa pur mantendo, data la sua leggerezza, prestazioni adeguate per il contesto del progetto.

3.2.1 Il Detector

Il detector ha il compito di individuare la posizione dei singoli caratteri Braille all'interno dell'immagine e fornire le corrispondenti *bounding box*, che verranno successivamente utilizzate per estrarre le singole celle da inviare al classificatore.

Poiché l'obiettivo di questa fase è esclusivamente la localizzazione dei caratteri e non la loro identificazione semantica, il modello è stato configurato in modalità *single-class detection*. In questo modo la rete deve semplicemente distinguere tra presenza e assenza di una cella Braille, riducendo la complessità del problema.

L'addestramento è stato condotto sul dataset composito descritto nella Sezione 3.1, nel formato richiesto dal framework Ultralytics. Le immagini di input sono state ridimensionate a 800×800 pixel. Come emerso dall'analisi geometrica condotta nella fase di EDA, tale risoluzione garantisce che ogni carattere Braille venga mappato su un numero accettabile di feature cell, offrendo il bilanciamento ottimale tra preservazione dei micro-dettagli spaziali ed efficienza computazionale.

Data la ridotta dimensione dei target, è stata inoltre valutata una possibile modifica architetturale mediante l'introduzione di una *P2 detection head*. Questa estensione, talvolta utilizzata nei problemi di *small-object detection*, consiste nell'aggiungere un ulteriore livello di predizione operante su feature map a risoluzione più elevata rispetto alle tradizionali scale utilizzate da YOLO. L'obiettivo è migliorare la capacità del modello di individuare oggetti molto piccoli.

Tuttavia, gli esperimenti preliminari condotti con questa configurazione non hanno mostrato miglioramenti significativi nelle metriche di valutazione del detector. Considerato inoltre l'aumento del costo computazionale e della complessità del modello introdotto da tale modifica, si è scelto di adottare la configurazione architetturale standard.

Alcuni parametri sono stati inoltre adattati per tenere conto della particolare densità dei caratteri Braille nelle immagini. In particolare, è stato aumentato il numero massimo di rilevamenti consentiti per immagine, da 300 a 1500, così da permettere al modello di individuare simultaneamente un

elevato numero di celle all'interno della stessa pagina.

Data augmentation Per migliorare la capacità di generalizzazione del modello rispetto a variazioni moderate nelle condizioni di acquisizione dell'immagine, durante l'addestramento sono state utilizzate le tecniche di data augmentation integrate nel framework Ultralytics. Le trasformazioni attive sono le seguenti:

1. Parametri configurati esplicitamente (Non-default)

Le seguenti trasformazioni sono state modificate o disabilitate rispetto al comportamento predefinito:

- **Flip:** L'inversione orizzontale (`fliplr=0.5`), che specchia l'immagine lungo l'asse x, è stata mantenuta attiva al 50% per generalizzare l'illuminazione proveniente da direzioni opposte. Al contrario, l'inversione verticale (`flipud=0.0`), che capovolge l'immagine lungo l'asse y, è stata esplicitamente disabilitata poichè distruttiva a causa delle caratteristiche di illuminazione del dataset.

- **Mosaic:** La Mosaic augmentation combina quattro immagini di addestramento in una sola, rivelandosi altamente efficace per migliorare il rilevamento di piccoli oggetti e la comprensione del contesto. Sebbene sia stata mantenuta attiva al 100% ($p = 1.0$), è stato introdotto il parametro `close_mosaic=20` per disabilitarla dinamicamente nelle ultime 20 epoche. Questo accorgimento permette al modello di eseguire un *fine-tuning* finale su immagini con proporzioni e scale reali.

- **Tecniche di fusione e cancellazione:** Moduli che miscelano due immagini (*Mixup*), cancellano casualmente porzioni dell'immagine (*Random Erasing*) o copiano oggetti tra immagini (*Copy-Paste*) sono stati completamente azzerati (`mixup=0.0`, `erasing=0.0`, `copy_paste=0.0`). Nel dominio OBR tali tecniche altererebbero la griglia 3×2, corrompendo le celle o generando pattern Braille inesistenti.

2. Parametri mantenuti a Default

Le seguenti trasformazioni sono state ereditate dalle impostazioni standard, in quanto ritenute efficaci e non distruttive per garantire invarianza spaziale e fotometrica:

- **Variazioni geometriche:** Sono state mantenute la scalatura casuale (`scale=0.5`), che ridimensiona le immagini di un fattore casuale, e la

traslazione (`translate=0.1`), che sposta le immagini orizzontalmente e verticalmente.

- **Variazioni fotometriche (HSV):** Modifiche alla tonalità (`hsv_h=0.015`), saturazione (`hsv_s=0.7`) e luminosità (`hsv_v=0.4`) sono applicate per simulare l'invecchiamento della carta, fotocamere con bilanciamenti del bianco errati e le diverse temperature dell'illuminazione ambientale.

- **Integrazione Albumentations:** Poiché la libreria *Albumentations* è installata nell'ambiente, Ultralytics applica automaticamente un set aggiuntivo di *augmentation*. Trasformazioni come sfocatura (*Blur*, *MedianBlur*), equalizzazione locale del contrasto (*CLAHE*) e conversione in scala di grigi (*ToGray*) vengono introdotte con probabilità molto bassa ($p = 0.01$). Queste alterazioni sono applicate appositamente a basse frequenze per imitare artefatti visivi del mondo reale.

3.2.2 Il classificatore

Scartata l'idea di utilizzare la rete YOLO26n anche per svolgere questo compito, è stata al suo posto inizialmente sviluppata una semplice rete CNN custom. I risultati ottenuti dal primo prototipo, basato su una classificazione diretta dei caratteri Braille in una delle 64 classi possibili, si sono rivelati insoddisfacenti.

L'analisi del problema ha evidenziato come la causa principale fosse il forte sbilanciamento nella distribuzione delle classi (descritto nella sezione 3.1). Alcuni caratteri risultano estremamente frequenti, mentre altri compaiono solo in poche istanze o sono completamente assenti. In tali condizioni un classificatore multi-classe tende inevitabilmente a privilegiare le classi più rappresentate, compromettendo la capacità di generalizzazione.

Per superare questa limitazione è stato adottato un approccio differente, che sfrutta direttamente la struttura geometrica del sistema Braille. Ogni carattere è infatti definito dalla presenza o assenza di sei punti disposti secondo una griglia 3×2. Di conseguenza, invece di trattare il problema come una classificazione a 64 classi, è possibile riformularlo come un problema di *multi-label binary classification*, in cui il modello deve semplicemente determinare la presenza o l'assenza di ciascuno dei sei punti.

In questo schema ogni punto viene rappresentato come una variabile binaria e l'output del classificatore consiste in un vettore di sei valori binari che descrive la configurazione del carattere. Tale rappresentazione può essere

interpretata come un codice a 6 bit, che identifica univocamente il carattere Braille e può essere successivamente convertito nel corrispondente simbolo testuale mediante una semplice operazione di mapping.

Il classificatore è basato su una rete convoluzionale leggera progettata su misura, chiamata **BrailleDotNet**. La rete riceve in ingresso l'immagine di una singola cella Braille (ritagliata nella fase intermedia posta tra detector e classificatore), convertita in scala di grigi e ridimensionata al formato fisso di 64×48 pixel. Come mostrato in Figura 3, l'elaborazione si divide in tre fasi principali:

1. **Stem:** Il primo strato della rete utilizza una convoluzione standard con un kernel di dimensioni ridotte (3×3). Questa configurazione funge da passaggio obbligato per espandere l'immagine di partenza (in scala di grigi, quindi a un singolo canale) in un volume multi-canale. Al contempo, un kernel piccolo permette di catturare immediatamente i contorni netti e le alte frequenze tipiche dei rilievi Braille, senza sfocare l'informazione visiva.

2. **Residual Blocks:** Il corpo centrale è stato progettato fondendo i principi architetturali di due modelli noti in letteratura: **MobileNet** e **ResNet**. Dalla prima si eredita l'utilizzo delle *depthwise separable convolutions*, una tecnica che abbatte drasticamente il costo computazionale e rende il modello idoneo ai dispositivi mobili. Dalla seconda mutua le connessioni residue (shortcut), essenziali per mantenere stabile l'addestramento negli strati più profondi. In questa fase, la rete estrae feature sempre più complesse.

3. **Flatten (Preservazione topologica):** Nelle reti di classificazione generiche, lo stadio finale prevede solitamente un *Global Pooling* per riconoscere un oggetto indipendentemente dalla sua posizione nello spazio. Nel dominio del Braille, tuttavia, la posizione esatta del punto veicola l'intero significato del carattere. Per questo motivo, tale operazione è stata intenzionalmente rimossa in favore di un appiattimento diretto (*Flatten*). Questa scelta di design costringe la rete a mantenere una mappatura rigida e deterministica rispetto all'esatta topologia geometrica della cella Braille.

L'ultimo strato è un *layer* lineare (protetto dall'overfitting tramite *Dropout*) che restituisce sei valori finali (*logits*), uno per ogni possibile punto della

cella Braille. Questi numeri indicano la probabilità che un certo punto sia in rilievo; applicando semplicemente una soglia (implementata al 50%), si ottiene infine la configurazione binaria (a 6 bit) del carattere letto.

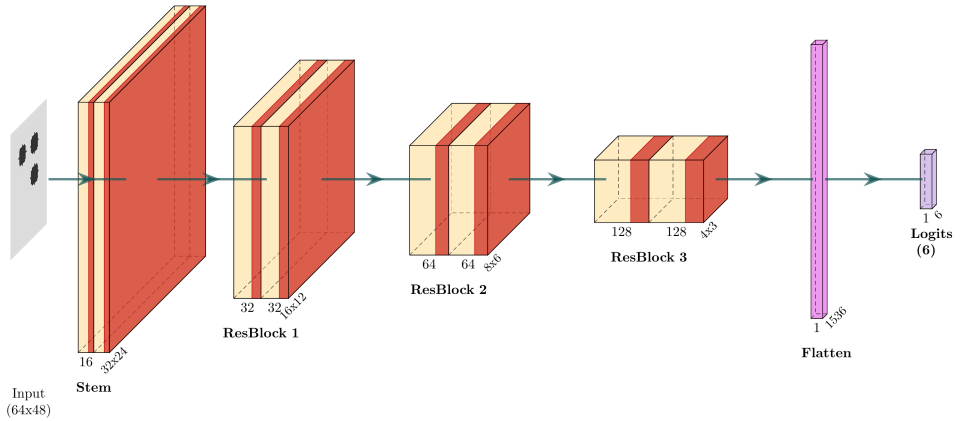

Figure 3: Architettura BrailleDotNet

Il dataset utilizzato per l'addestramento del classificatore è stato costruito a partire dai ritagli delle celle Braille ottenuti dal dataset principale, effettuandone uno specifico preprocessing ed una data agumentation come vengono descritti in dettaglio di seguito. A questi sono stati aggiunti esempi negativi, provenienti da immagini che non contengono caratteri Braille (presi dal Dataset Angelina) e da punti appartenenti al lato opposto della pagina (verso) che non interferiscono con i caratteri presenti sul recto (presi dal dataset DSBI).

La loro inclusione consente al classificatore di migliorare la propria robustezza complessiva e contribuendo a compensare eventuali falsi positivi generati dal detector.

Preprocessing e Normalizzazione Il passaggio dalle *bounding box* ai ritagli per il classificatore avviene attraverso una pipeline in cinque step, studiata per non deformare i punti Braille:

1. **Margine:** Il box predetto viene allargato del 15% su ogni lato per includere una porzione di sfondo intorno ai punti.

2. **Correzione geometrica:** Per evitare di "schiacciare" i punti Braille nel passaggio successivo, si forzano le coordinate del box ad un rapporto di $3:4$ (lo stesso richiesto dalla rete). Se il box è troppo stretto o troppo basso, viene allargato matematicamente il lato più piccolo.

3. **Conversione, ritaglio e padding:** L'immagine viene convertita in scala di grigi e viene eseguito il ritaglio. Se il box finisce fuori dai bordi dell'immagine originale, la parte mancante viene riempita con il colore medio dell'aerea circostante (*Local Background Padding*), evitando di creare bordi neri o artificiali.

4. **Ridimensionamento:** L'immagine ottenuta viene portata alla risoluzione finale di 64×48 pixel. Si utilizza l'interpolazione *Bicubica* per gli ingrandimenti e quella ad *Area* per le riduzioni, garantendo la massima nitidezza dei rilievi.

5. **Normalizzazione:** I pixel vengono normalizzati nell'intervallo $[-1, 1]$.

Data Augmentation La Data Augmentation viene applicata dinamicamente in memoria tramite la libreria Albumentations:

- **Flip orizzontali:** Viene applicato casualmente un *Horizontal Flip*. Poiché il Braille non è simmetrico, specchiando l'immagine la rete rimappa dinamicamente anche il tensore bersaglio, invertendo le etichette della colonna di sinistra con quelle di destra.

- **Trasformazioni affini:** Variazioni minime di scala ($\pm 5\%$), traslazione ($\pm 5\%$) e rotazione ($\pm 5°$). Queste alterazioni simulano l'inevitabile imprecisione dei box del detector a monte e il naturale tremolio della fotocamera.

- **Rumore e sfocatura:** Aggiunta occasionale di sfocatura gaussiana (*Gaussian Blur*) e rumore di sensore (*Gauss Noise*). Questi filtri abituano il modello a decifrare il Braille anche in fotografie fuori fuoco o scattate con smartphone di bassa qualità in condizioni di scarsa illuminazione.

- **Variazioni fotometriche:** Alterazioni casuali di luminosità e contrasto ($\pm 30\%$) per simulare l'effetto di ombre proiettate, riflessi o supporti di colore o spessore differente.

3.3 Il modello finale

La pipeline completa comprende quindi tre fasi principali:

1. **Detector**: il modello YOLO26n riceve l'immagine in ingresso e individua la posizione delle celle Braille presenti nella pagina, producendo come output l'insieme delle relative *bounding box*.

2. **Preprocessing**: a partire dalle bounding box generate dal detector vengono estratte le singole celle Braille. Ogni ritaglio viene quindi sottoposto a una serie di operazioni di preprocessing, come meglio descritto in 3.2.2. Tra cui conversione in scala di grigi, resize e padding, al fine di ottenere un input uniforme per la rete di classificazione.

3. **Classificatore**: il modello BrailleDotNet, per ciascuna cella preprocessata fornita, predice la presenza o l'assenza dei sei punti che compongono il carattere Braille. L'output della rete consiste quindi in un vettore binario a sei bit che identifica univocamente il carattere.

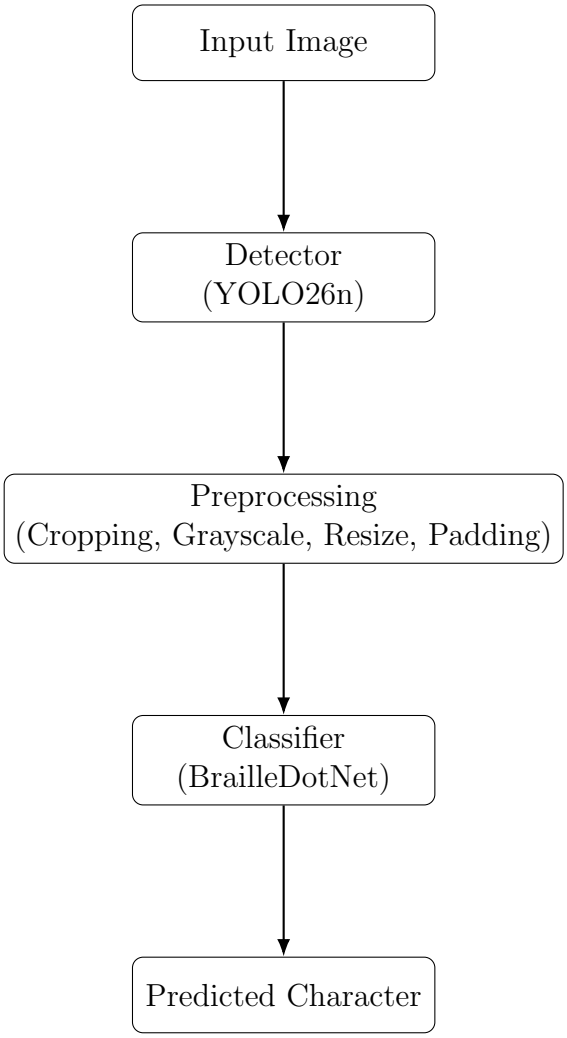

Figure 4: Pipeline completa del sistema proposto

4 Valutazione Sperimentale

L'addestramento dei modelli è stato condotto su hardware diverso. Per il detector si è sfruttato l'hardware fornito dall'ambiente cloud Google Colab (piano gratuito), impiegando in particolare una GPU NVIDIA Tesla T4 dotata di 15 GB di VRAM. Mentre per il classificatore è stata usata una GPU NVIDIA GTX 1050M dotata di 3 GB di VRAM.

Data la struttura modulare del modello e il rischio di propagazione d'errore,

risulta opportuna una valutazione separata per il detector e per il classificatore. Le metriche di cui tenere conto sono altresì differenti per le due reti, in quanto progettate con scopi diversi.

Nelle sezioni che seguono verranno analizzati separatamente i risultati delle due parti e poi quelli complessivi. Verranno riportate le medesime metriche al fine di agevolare eventuali paragoni con la letteratura, ma verrà posto risalto solamente su quelle di fondato interesse. Per dimostrare la capacità di generalizzazione del sistema, le tabelle riporteranno un confronto diretto delle performance sia sul validation che sul test set.

4.1 Valutazione del detector

Lo scopo del detector è individuare i caratteri Braille all'interno dell'immagine: il sistema è da ritenersi valido se riesce a localizzarli tutti e a inquadrarli con una precisione geometrica sufficiente a permetterne la successiva classificazione.

Per raggiungere questo obiettivo, il modello YOLO26n è stato addestrato per un massimo di 150 epoche (con meccanismo di *early stopping* impostato a 40 epoche), con un *batch size* di 8 e una risoluzione delle immagini di 800×800 pixel. Come ottimizzatore è stato impiegato MuSGD con i parametri standard forniti dal framework Ultralytics. Coerentemente con la natura del problema, è stata attivata la flag *single-class* ed è stato assegnato un peso superiore alla funzione di costo per la regressione delle *bounding box* (*box=10.0*).

La metrica più informativa in questa fase è la mAP50: un rilevamento con IoU ≥ 0.5 garantisce che una porzione sostanziale del carattere sia stata delimitata. Normalmente, in architetture a cascata, sarebbe indispensabile accertarsi che il *bounding box* risulti perfettamente centrato affinché il classificatore possa operare in condizioni ottimali. Tuttavia, grazie alle scelte di design adottate in fase di pre-elaborazione — nello specifico l'espansione preventiva del margine del ritaglio — e alla robustezza indotta dalla *data augmentation*, il modello a valle è in grado di tollerare lievi imprecisioni geometriche. Finché i punti che compongono il carattere Braille sono interamente racchiusi all'interno della *bounding box* e non subiscono amputazioni (evitando la perdita fisica di informazione), il classificatore è in grado di compensare il disallineamento e decodificare correttamente il simbolo.

Per questa ragione, la mAP50 risulta più indicativa della mAP50:95 ai fini del corretto funzionamento del sistema. È stata inoltre inclusa la mAP75

come metrica di robustezza: un valore elevato in questo parametro convalida la qualità del detector anche in scenari in cui la media su diverse soglie IoU risulti inferiore.

L'altra metrica cruciale è il *Recall*: un valore basso in questa metrica rappresenta la principale fonte di propagazione d'errore del sistema, poiché un carattere non rilevato non può essere classificato. La priorità è stata data al *Recall* rispetto alla *Precision* per finalità pragmatiche: mentre un falso positivo (una porzione di sfondo scambiata per un carattere) può essere successivamente scartato dal classificatore come spazio vuoto, un falso negativo comporta una perdita irrimediabile di informazione.

Metrica	**Validation Set**	**Test Set**
Precision	0.9957	0.9782
Recall	0.9928	0.9628
F1-score	0.9943	0.9705
mAP@0.5	0.9946	0.9826
mAP@0.75	0.9772	0.6182
mAP@0.5:0.95	0.8718	0.5935

Table 2: Performance globali del Detector: confronto tra Validation e Test Set.

Alla luce di quanto premesso, i dati in Tabella 2 mostrano che il detector generalizza in modo piuttosto solido su dati mai visti. Sul *Test Set*, il *Recall* si mantiene ad un buon 96.28%, assicurando una minima dispersione di informazioni. La mAP@0.5 decresce solo marginalmente al 98.26%, confermando che il modello inquadra correttamente la quasi totalità dei target.

Tuttavia, emerge un calo marcato nella precisione geometrica più stringente (mAP@0.75), che scende dal 97.72% del *Validation Set* al 61.82% del *Test Set*. Per comprendere la natura di questa flessione, è fondamentale esaminare la scomposizione per dominio riportata nella Tabella 3.

Il dato più critico riguarda il dominio Angelina sul *Test Set*, dove la mAP@0.75 crolla a 0.0251. Ciò indica che, sebbene il modello individui le celle (Recall 92.22%), le *bounding box* predette presentano un disallineamento geometrico sistematico rispetto alle annotazioni. Questa risulta effettivamente veritiero, come osservto nella fase di EDA dove risulta che mediamente le celle braille del test set sono più piccole rispetto a quelle presenti nel train e nel val set. Si tratta di un caso esemplare di *Domain Shift*: le

Dominio	**Recall**		**mAP@0.5**		**mAP@0.75**	
	Val	*Test*	*Val*	*Test*	*Val*	*Test*
Angelina	0.9948	0.9222	0.9949	0.9505	0.9936	0.0251
Natural Scenes	0.9774	0.9602	0.9931	0.9895	0.9775	0.9769
DSBI	0.9918	0.9811	0.9948	0.9943	0.7508	0.9515

Table 3: Scomposizione delle performance del detector per dominio.

immagini di test di questo dataset provengono da sorgenti diverse da quelle di addestramento, con parametri di spaziatura o inquadratura che la rete non ha perfettamente generalizzato.

Nel tentativo di risolvere questa problematica si è cercato di addestrare il modello sfruttando un Multi-Scale Training. Tuttavia, l'addestramento non è mai stato portato a termine a causa delle limitazioni hardware che portavano alla continua generazione di errori di tipo Out-Of-Memory. Pertanto si è deciso di mantenere la risoluzione fissa a 800×800 pixel.

4.2 Valutazione del classificatore

Lo scopo del classificatore è riconoscere correttamente il carattere ricevuto in input. Per fare ciò, il modello deve determinare con precisione lo stato di ognuno dei sei punti che compongono la cella Braille.

L'addestramento della rete è stato effettuato con un *batch size* di 512. Come ottimizzatore è stato impiegato AdamW (*learning rate* massimo di 0.005 e *weight decay* di 10^{-4}), modulato da uno *scheduler* OneCycleLR per garantire una convergenza rapida. Coerentemente con la formulazione del task come *multi-label binary classification*, è stata adottata la loss function *Binary Cross-Entropy with Logits* (BCEWithLogitsLoss). L'addestramento è stato configurato per un massimo di 35 epoche, con un meccanismo di *early stopping* impostato a 15 epoche.

La metrica più severa e rilevante per la valutazione finale è la *CharAccuracy*, poiché un singolo errore su uno dei sei punti (falso positivo o falso negativo) compromette la coretta classificazione del carattere. A questa metrica si affiancano la *CharPrecision* e la *CharRecall*. La prima quantifica l'affidabilità delle predizioni, indicando la percentuale di caratteri classificati correttamente rispetto al totale delle predizioni per quella classe (limitando

così i falsi positivi). La seconda valuta invece la sensibilità del sistema, ossia la sua capacità di individuare e classificare correttamente i caratteri realmente presenti nell'immagine (minimizzando i falsi negativi). Un aspetto metodologico cruciale è l'utilizzo dell'aggregazione *Macro* per entrambe le metriche: calcolando le prestazioni per ogni singola classe e facendone una media non pesata, si garantisce che gli errori sui caratteri più rari non vengano nascosti statisticamente dalle ottime performance sulle lettere più frequenti. Infine vi è la *Char F1-score*, anche questa Macro, che fonde precisione e sensibilità. Penalizzando severamente i modelli che presentano un forte divario tra queste due metriche, un punteggio elevato certifica una capacità di generalizzazione solida e omogenea su tutte le classi.

A completamento dell'analisi è stata introdotta la *DotAccuracy*, che valuta l'accuratezza della predizione a livello di singolo punto fisico (Dot 1-6), indipendentemente dal carattere a cui appartiene. Questa metrica assume un ruolo diagnostico fondamentale: un valore di *Dot Accuracy* nettamente superiore alla *Char Accuracy* dimostra che i fallimenti della rete non derivano da un'incomprensione globale dei *pattern* visivi, ma sono per lo più circoscritti a isolate incertezze su singoli bit locali.

Metrica (Overall)	**Validation Set**	**Test Set**
Dot Accuracy	0.9989	0.9960
Char Accuracy (EMR)	0.9952	0.9843
Char Precision (Macro)	0.9864	0.9783
Char Recall (Macro)	0.9869	0.9781
Char F1-score (Macro)	0.9864	0.9780

Table 4: Performance globali del Classificatore BrailleDotNet.

Le metriche globali riportate nella Tabella 4 confermano la solidità della scelta architetturale. Il passaggio dal *Validation* al *Test Set* mostra una flessione minima, con una *Char Accuracy* complessiva che sfiora il 98.5%. Per un'analisi più approfondita della capacità di generalizzazione del modello, la Tabella 5 riporta le prestazioni disaggregate per i tre domini principali che compongono il dataset.

Dall'analisi disaggregata per dominio emergono considerazioni interessanti.

Per quanto riguarda il dataset *Angelina*, si osserva un calo della *Char Accuracy* che scende a 94.06% e della *Macro F1-score* che subisce una flessione

Dominio	**Char Accuracy (EMR)**		**Char F1-score**		**Dot Accuracy**	
	Val	*Test*	*Val*	*Test*	*Val*	*Test*
Angelina	0.9989	0.9406	0.9418	0.8340	0.9998	0.9835
Natural Scenes	0.9416	0.9540	0.8472	0.9034	0.9849	0.9862
DSBI	0.9937	0.9960	0.9758	0.9777	0.9989	0.9993

Table 5: Scomposizione delle performance del classificatore per dominio.

fino allo 83.40%. Questo comportamento, in perfetta continuità con quanto già evidenziato durante la valutazione del detector, conferma che le immagini di test possiedono caratteristiche visive e tipografiche differenti rispetto a quelle di addestramento, innescando un tangibile *Domain Shift*.

In conclusione, l'approccio basato sulla scomposizione *multi-label* a 6 bit non solo ha mitigato con successo lo sbilanciamento delle classi, ma ha permesso di ottenere un sistema di classificazione estremamente robusto, interpretabile e resiliente alle variazioni di dominio.

4.3 La valutazione complessiva

In questa fase conclusiva, il sistema viene valutato nella sua interezza come un'unica *pipeline end-to-end*. Affinché una predizione venga considerata corretta (TP), il sistema deve soddisfare due criteri simultaneamente: il *detector* deve localizzare la cella con un'intersezione (IoU) ≥ 0.5 e il classificatore deve indovinare l'esatta configurazione dei 6 bit. In questo scenario, le metriche usate sono state calcolate in modalità Micro aggregando globalmente le predizioni sull'intero dataset anziché mediare le singole classi. Nello specifico, la *Recall* quantifica la capacità del sistema di non omettere informazioni, mentre la *Precision* ne misura l'affidabilità penalizzando le allucinazioni causate dal rumore di fondo. L'equilibrio tra sensibilità e specificità è sintetizzato dalla *F1-score*, che offre una stima di robustezza globale. Infine, l'analisi è completata dalla *Strict Accuracy* ($TP/(TP + FP + FN)$), la metrica più severa in assoluto: essa rappresenta la percentuale di caratteri perfettamente localizzati e decodificati rispetto al volume totale di predizioni generate e target reali.

Al fine di quantificare la reale capacità di generalizzazione dell'architettura sui diversi domini visivi, la Tabella 6 illustra la diagnostica dettagliata sui *Validation* e *Test Set*.

Dominio	**Precision**		**Recall**		**F1-Score**		**Strict Acc**	
	Val	*Test*	*Val*	*Test*	*Val*	*Test*	*Val*	*Test*
Angelina	0.9735	0.8613	0.9975	0.8684	0.9854	0.8649	0.9711	0.7619
Natural Scenes	0.8514	0.8274	0.9071	0.9009	0.8784	0.8626	0.7831	0.7584
DSBI	0.8933	0.9306	0.9239	0.9875	0.9084	0.9582	0.8321	0.9198
Overall	**0.9558**	**0.9110**	**0.9826**	**0.9546**	**0.9690**	**0.9323**	**0.9398**	**0.8732**

Table 6: Metriche della *pipeline end-to-end*: confronto tra Validation e Test Set per dominio.

I risultati evidenziano come strutture *two-stage pipeline* di questo tipo soffrano della propagazione d'errore. La scomposizione per dominio conferma le dinamiche già osservate nei singoli moduli: il dominio *DSBI* si rivela il più performante in assoluto in fase di test, mentre il dataset *Angelina* subisce un fisiologico degrado dovuto al *Domain Shift* precedentemente discusso.

4.3.1 Benchmarking di Riferimento (Reference SOTA)

È fondamentale premettere che un confronto numerico diretto (1:1) con lo stato dell'arte risulta metodologicamente improprio. Le metriche riportate in letteratura, seppur eccellenti, sono calcolate in condizioni sperimentali non sovrapponibili a quelle di questo lavoro, per tre ragioni fondamentali:

In primo luogo, vi è una profonda differenza nel **tipo di metrica**. Alcuni studi recenti riportano valori altissimi (es. l'F1-score del 98.00% di Valencio et al. [7] o il 96.60% di Janrao et al. [1]), ma tali metriche si riferiscono esclusivamente alla fase di classificazione su singole celle Braille già perfettamente ritagliate e centrate, ignorando del tutto l'errore cumulativo che si genera localizzando i target sull'intera pagina. Il valore ottenuto di F1-score pari a 93.23%, al contrario, è una severa valutazione *End-to-End*, che penalizza il sistema per ogni minima imprecisione geometrica o semantica sull'intero documento.

In secondo luogo, la **dimensione e l'eterogeneità del Test Set** differiscono drasticamente. L'incredibile F1-score del 99.81% raggiunto da Ovodov (2021) [4] è stato calcolato su uno *split* di appena 49 immagini, mentre il 99.29% di Li et al. (2020) [2] si basa su sole 30 immagini di test. Il nostro *MergedDataset* ha invece validato la rete su 370 immagini provenienti da domini diversi e da fonti diverse, esponendo il modello a un *Domain Shift* severo che alterna continue deformazioni della carta a rumore di supporti

differenti.

Infine, il **costo computazionale**. Le architetture *single-stage* o di segmentazione semantica ai vertici della letteratura (come RetinaNet o BraU-Net) sono modelli estremamente pesanti, la cui inferenza in letteratura viene calcolata avvalendosi di potenti GPU desktop (es. NVIDIA GTX 1080Ti). La *two-stage pipeline* proposta invece è caratterizzata da un *detector* di circa 2M di parametri che insieme ad una CNN custom garantisce un *footprint* compatibile con il *deployment real-time* su dispositivi mobili e *edge*.

Pertanto, la Tabella 7 non rappresenta una competizione diretta, bensì uno strumento qualitativo per contestualizzare le prestazioni del modello rispetto a quelle raggiunte in letteratura in domini isolati.

Autore (Anno)	**Architettura**	**Tipo di Valutazione**	**Dataset di Test**	**F1-Score**
Questo Modello	**YOLO26n + CNN Custom**	**End-to-End**	**MergedDataset** (> 1500 **img**)	**93.23%**
Ovodov (2021) [4]	RetinaNet (Single-stage)	End-to-End	Angelina (Split 49 img)	99.81%
Li et al. (2020) [2]	BraU-Net (Segmentation)	End-to-End	DSBI (Split 30 img)	99.29%
Valencio et al. (2025) [7]	YOLOv8 + MobileNetV2	Solo Classificazione	Custom Mix	98.00%
Janrao et al. (2025) [1]	DenseNet	Solo Classificazione	Custom Dataset	96.60%
Sun et al. (2025) [6]	YOLOv11 (ULSAM)	End-to-End	Natural Scene (Split 111 img)	94.67%

Table 7: Contesto di riferimento dello Stato dell'Arte (SOTA)

Oltre all'accuratezza semantica, la vocazione *lightweight* del sistema è stata validata attraverso un'analisi della latenza di inferenza. Sebbene un confronto diretto risulti metodologicamente complesso a causa della diversità dell'hardware tra i vari studi, i dati raccolti evidenziano un netto vantaggio computazionale dell'architettura proposta.

La *pipeline* senza particolari ottimizzazioni ha registrato un tempo di inferenza medio sul *Test Set* di **38,21 ms** per immagine (equivalenti a circa **26,17 FPS**) operando su una GPU NVIDIA GTX 1050M. In termini comparativi, i modelli *single-stage* o basati su segmentazione presenti in letteratura riportano latenze significativamente superiori, pur avvalendosi di hardware di classe *desktop* nettamente più performante: Ovodov (2021) dichiara infatti 180 ms per immagine [4] e Li et al. (2020) circa 250 ms [2], entrambi su NVIDIA GTX 1080Ti. Anche studi recenti focalizzati sul tempo reale, come quello di Valencio et al. (2025), attestano prestazioni comprese tra 5 e 7 FPS su piattaforme *laptop* [7].

5 Conclusioni

Il progetto ha dimostrato una buona efficacia generale, raggiungendo parzialmente gli obiettivi prefissati. Le prestazioni migliori del sistema rimangono tuttavia legate a condizioni esterne favorevoli, come la corretta distanza dalla fotocamera e un'adeguata angolazione della luce. L'architettura sviluppata si distingue per l'elevata velocità di calcolo ($\sim$ 38 ms su una GPU di fascia bassa), per l'approccio a 6 bit che permette di riconoscere correttamente anche i caratteri meno comuni e per la capacità di adattarsi a diversi scenari grazie all'impiego del *MergedDataset*.

Sebbene i limiti del dataset rendano il modello ancora sensibile ai cambiamenti di illuminazione e alla dimensione delle celle, questo progetto getta le basi per un utilizzo concreto su smartphone. Eventuali futuri sviluppi potranno beneficiare di dataset più avanzati che includano diverse angolazioni di luce radente, seguendo l'approccio proposto da [8]. Questo permetterà di rendere il sistema più stabile e pronto per un *deployment* definitivo tramite formati ottimizzati come *ONNX* o *TensorRT*, trasformando *BrailleLens* in uno strumento affidabile per l'assistenza nella vita quotidiana.

References

[1] Surekha Janrao, Tavion Fernandes, Ojas Golatkar, and Swaraj Dusane. Deep learning approaches for braille detection and classification: comparative analysis. *IAES International Journal of Artificial Intelligence (IJ-AI)*, 14(6):4652–4660, 2025.

[2] Renqiang Li, Hong Liu, Xiangdong Wang, Jianxing Xu, and Yueliang Qian. Optical braille recognition based on semantic segmentation network with auxiliary learning strategy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 554–555, 2020.

[3] Vishal V. Mahale, Shriniwas J. Patil, Lavkesh S. Patil, and Bhupesh A. Shinde. A survey on braille character recognition and text-to-speech conversion systems. In *Proceedings of the International Conference on Cognitive Computing and Information Processing*. Springer, 2022.

[4] Ilya G Ovodov. Optical braille recognition using object detection

cnn. *arXiv preprint arXiv:2012.12412*, 2020. Dataset available on: https://github.com/IlyaOvodov/AngelinaDataset.

[5] Mohammad Abdullah Sayeedi, Anas Mohammad Muktadir Osmani, and Dewan Md Farid. Braillesense: Deep learning for braille character classification. In *2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT)*, pages 681–686. IEEE, 2024.

[6] Yu Sun, Wenhao Chen, Yihang Qin, Xuan Li, and Chunlian Li. Real-time braille image detection algorithm based on improved yolov11 in natural scenes. *Applied Sciences*, 15(18):10288, 2025.

[7] Tan Valencio, Yobert Geraldo Setiawan, and Eko Hari Rachmawanto. A two-stage braille recognition system using yolov8 for detection and cnn for classification. *Journal of Applied Informatics and Computing (JAIC)*, 9(6):3057–3067, 2025.

[8] Akihiro Yamashita, Hiroki Furukawa, Taichi Shirakawa, and Katsushi Matsubayashi. Braille character recognition independent of lighting direction using object detection and segmentation models. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 30(1):222–231, 2026.